PA195

A 195-state binary Turing machine for the consistency of Peano arithmetic

Research prepared for Vladimir Reshetnikov

13 November 2026

Source snapshot: `906367250a7f96c923147b10df392294bced7be5`

Abstract

We give an explicit deterministic single-tape, two-symbol Turing machine with 195 working states, started on a two-way infinite all-zero tape, and an ordinary mathematical proof that Peano arithmetic proves its nonhalting equivalent to Con(PA). The construction preserves the 191-item diagonal-vacuity axiom emitter in the supplied research corpus. A new binary association admits seven simultaneously valid history-sensitive reader identifications: 134 inherited controls plus 68 reader controls, less seven, gives 195.

The packet contains the complete 390-row table, deterministic reconstruction, all 3,700 source instructions, 28 finite local simulation certificates, an exact 58,154-step startup certificate, and executable symbolic checks for all 4,815 primitive branches. An independent audit checks the source graph, two representations of startup, all protected engine rows, and 19,260 concrete instruction frames. The comparison covers all 27 incoming archives, the direct PA research directory, 20 designated U208 graph classes, and 25 distinct supported 196-state layouts. The new arithmetic endpoint is proved in ordinary mathematical form; a new end-to-end Lean or Rocq theorem is not claimed.

195 working states **2 symbols** **390 transitions**

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(\mathrm{PA195}, \mathrm{blank}) \;\leftrightarrow\; \mathrm{Con}(\mathrm{PA}).$$

Contents

1 Result and exact machine conventions **3**
1.1 What is new . . . 3
1.2 Meaning of the proof claim . . . 3

2 Comparison of the supplied designs **4**
2.1 Pinned scope and deduplication . . . 4
2.2 The 196-state frontier . . . 5
2.3 Why the older reductions are not additive . . . 5
2.4 Ideas reused from ZFC and ConciseIPRA . . . 6

3 Exact source and compilation **6**
3.1 Source representation and invariant interfaces . . . 6
3.2 The emitted numerical calculus . . . 7
3.3 Exact startup and total primitive simulation . . . 7

4 The seven-state quotient: a global finite certificate **8**
4.1 The interacting five-step case . . . 9

5 The PA-provable consistency equivalence **9**
5.1 The numerical calculus and its diagonal replacement . . . 10
5.2 A uniform translation of the numerical calculus into PA . . . 12
5.3 The exact proof-store invariant . . . 14
5.4 Why the finite runtime certificates are sufficient in PA . . . 15
5.5 Completing the other direction . . . 16
5.6 Logical strength and the role of ConciseIPRA . . . 17

6 Verification, reproduction, and evidence limits **17**
6.1 Reproduction without a search . . . 17
6.2 What the packet establishes . . . 18
6.3 Files and provenance . . . 18

A Complete transition table **19**

B Archive-by-archive comparison **20**

1 Result and exact machine conventions

The deliverable `pa195/pa195.tm` is the definitive literal machine. Its working labels are $0, \ldots, 194$, its start label is 0, and its separate halt label is 195. The tape alphabet is $\{0, 1\}$, the tape extends without bound in both directions, the initial head coordinate is zero, and every initial cell is zero. There is one transition for every working-state/read-symbol pair. No stay-put move is used.

A row (q, b, w, D, q') means: read b, write w, move one cell in direction $D \in \{L, R\}$, and enter q'. The halt control has no outgoing row and is excluded from the working-state count. All 195 working states are reachable in the finite transition graph. This last claim concerns graph reachability, not occurrence on the actual blank run.

Object	Exact value
Working states	195
Transition rows	390
Start / halt	0 / 195
Source root in the dense table	75
Unique halt transition	63, 1 $\mapsto$ 1, R, 195
First transition on blank	0, 0 $\mapsto$ 1, R, 1
Source instructions	3,700
Primitive branch contracts	4,815
Exact startup length	58,154 literal transitions

The SHA-256 of the literal five-column table, including its final newline, is

`5c6afd87f7c4f24d9ad8e9e302cc21df14b6f2e3715d03322a75463a431d8455`.

Appendix A prints the entire table in paired-column form. The native-label version and its complete conjugacy are also included; native label 254 and dense label 195 denote the same halt control.

1.1 What is new

The new result is a physical compilation of the existing *diagonal191* program below the 196-state threshold. Its theory, formula encoding, axiom order, formula parameters, register assignment, and exact ordered instruction sequence are unchanged from the `diagonal196` construction in the supplied archive [4]. The source parenthesization and reader allocation change. The ordinary PA proof in Section 5 explicitly connects this unchanged calculus to the delivered table.

For the supplied `diagonal196` parent, the allocation is $134 + 68 - 6 = 196$. For the new association, it is

$$\boxed{134 + 68 - 7 = 195.}$$

The new association therefore retains the same number of allocated readers before folding but admits one additional *compatible* fold. Optimizing only the number of syntax nodes would miss this saving. The 134 inherited controls comprise 36 engine controls, 13 leaf routers, and 85 readers outside this AX allocation. The final reader census is $85 + (68 - 7) = 146$; hence $36 + 13 + 146 = 195$.

1.2 Meaning of the proof claim

The theorem is the internal PA biconditional, not merely external nonhalting under the standard interpretation. It is proved here by uniform syntactic proof transformations, arithmetic induction, and

positive finite simulation. Its validity does not assume Con(PA). Actual blank-tape nonhalting follows if PA is consistent.

The published PA196 Lean development supplies the internal implication from nonhalting to consistency, together with its other external and independence results [1, 3]. It should not be described as already containing the converse internal implication. This report supplies the converse by syntactic PA proof annotations and finite bit-comprehension proof generation. The executable machine certificates and the ordinary proof are available for review; a newly elaborated Lean or Rocq endpoint for PA195 remains future formalization work.

2 Comparison of the supplied designs

2.1 Pinned scope and deduplication

The comparison fixes ProveIt commit `906367250a7f96c923147b10df392294bced7be5`. The direct `Computability/BusyBeaver/Research/PA` directory contains the imported PA196 and PA209 projects plus their comparison and source experiments [2]. The incoming directory has 27 ZIP archives, totaling 224,873,862 bytes, with 21,003 non-directory outer members. Text, code, and machine members were inspected or indexed; nested archives were inspected to depth two. Appendix B gives a contribution/status entry for every incoming archive.

The numeric table census finds 1,857 `.tm` occurrences and 521 distinct literal byte strings. These are *file* counts, not claims of 521 correct consistency machines: they include repeated ancestors, experimental allocations, rejected candidates, and small test controls. For meaningful comparisons we canonicalize designated tables as rooted, read-symbol- and action-preserving graphs. A canonicalization preserves the start control, the halt control, writes, directions, and the two read-symbol labels; it does not collapse a change of tape convention.

The designated U208 corpus has 27 presentations and 20 graph classes. The first nine presentations form two classes, with five and four presentations respectively. Eighteen later helper variants each add a class. Twenty-five distinct supported 196-state layouts are retained. Twenty-three have complete evidence in the supplied reports; two more have their missing startup or contract checks completed in this audit. *Supported* means that the table has its stated source, runtime, startup, and mathematical evidence. It does not upgrade every report to a kernel-checked theorem.

The full 196-state census contains 268 table occurrences, 131 distinct byte strings, and 90 rooted graph classes. Every class is classified in the accompanying ledger; the remaining classes have partial, bounded, or negative evidence and are not silently promoted to the supported frontier.

The incoming corpus also contains 14 occurrences of 195-state tables: eight literal byte strings and six candidate constructions after native/dense renamings are identified. All are rejected or uncertified in their source reports. Some fail an exact primitive contract; others have a wrong source successor or an explicit blank-tape halt. They are not predecessors already meeting the requested endpoint.

2.2 The 196-state frontier

Design	States	Sites	Contracts	Startup	Evidence
Published PA196	196	3,812	4,959	58,154	K
Alternative head order	196	3,812	4,959	58,154	E
Closing CD3	196	3,812	4,959	58,154	I
Closing CD4	196	3,812	4,959	58,154	I
Closing CD5	196	3,812	4,959	58,154	I
Closing CD6	196	3,812	4,959	58,154	I
Rotation layout 1	196	3,812	4,959	58,294	E
Rotation layout 2	196	3,812	4,959	58,294	E
Eight-zero scanner layout	196	3,812	4,959	59,164	E
A/D register swap	196	3,812	4,959	58,122	E
Rebracketed source	196	3,812	4,959	58,154	E
H-only Shift	196	3,811	4,958	57,922	E
Twelve-rotation layout	196	3,812	4,959	58,294	E
Combined196	196	3,788	4,928	57,922	E
Diagonal196	196	3,700	4,815	58,154	E+L
Separate196	196	3,673	4,780	58,154	E+L
Separate/scanner196	196	3,673	4,780	58,294	E+L
Original/scanner196	196	3,812	4,959	58,294	E
Zerojump196	196	3,812	4,959	58,402	I
Diagonal closing CD3	196	3,700	4,815*	58,154	I+L
Diagonal closing CD4	196	3,700	4,815*	58,154	I+L
Diagonal closing CD5	196	3,700	4,815*	58,154	I+L
Diagonal closing CD6	196	3,700	4,815*	58,154	I+L
Zerojump CD3	196	3,812	4,959	58,366	I†
Rebracketed Combined196	196	3,788	4,928	57,922	E†
New PA195	195	3,700	4,815	58,154	E+L

Evidence key. K: the published Lean development, including its internal implication from nonhalting to consistency. E: complete executable symbolic runtime and exact startup, with ordinary mathematical transfer. I: runtime uses a proved source invariant. L: the numerical calculus changes and therefore needs its stated logical calibration. None of E, I, or L denotes a newly compiled kernel theorem. The new PA195 row has a full ordinary proof of the internal biconditional in this report.

*For each diagonal closing variant, 45 affected branches are replayed as 69 invariant-restricted affine cases; every other baseline branch avoids the changed row. This differential proof covers all 4,815 baseline branches.

†The CD3 zerojump and rebracketed Combined196 tables are archived designs whose missing complete contract or startup check was supplied in this audit. Their fresh receipts are included. The source counts are primitive instruction occurrences. Contract counts include both outcomes of a conditional decrement. Prior numerical figures are supported by the supplied receipts, with the two audit-completed cases marked separately; the PA195 row is newly recomputed. Exact archive/member/hash bindings are in `comparison/frontier196_and_pa195.json`, and complete state bijections for the 25 normalized tables are retained under `comparison/prior196/`.

2.3 Why the older reductions are not additive

James Chang's specified 209-state predecessor and the U209 parents of the U208 archives are distinct machines. The U208 saving identifies the particular native roles 172 and 211 using entry history and

an existing helper. Those old roles are already absent from PA196. One cannot simply subtract that saving again from a different 196-state table.

The published 209-to-196 change combines the transposed quantifier code, a smaller equality interface, changed axiom order, and contextual reader sharing [1]. The original PA196 has 3,812 source sites and 159 distinct internal source nodes; its 201-item AX word uses 68 allocated reader nodes and six folds. The diagonal-vacuity source reduces the AX word to 191 and the full source to 3,700 sites. Its earlier physical layouts reached 196, and provide the parent used here.

Several apparently simpler proposals do not yield a smaller binary machine. A 157-node source can lose a required helper. A different 158-node layout needs an extra scanner state. The five-zero source representation changes the initial source address and loses inherited contextual sharing. The separate-vacuity emitter has only 190 macro items, but pushes ten stack entries instead of nine and has a different command compiler interface. These examples explain why the present search scores an actual compatible allocation.

The direct PA directory's additional literal tables are inventoried in `comparison/pa_directory_inventory.json`. Besides the two published tables and alternative196, these include 197-state negation/MP and shortcut experiments, source-pruning allocations, an identity reconstruction, and a 198-state root-frame experiment with an unsettled blank initialization. None supplies a smaller accepted table.

2.4 Ideas reused from ZFC and ConciseIPRA

The useful ZFC inheritance is the compact unary-register engine and its symbolic block contracts, source-tree sharing, and history-sensitive reader reuse [6]. The new machine keeps this engine and changes its compiled proof-search program representation.

The useful proof-theoretic idea is to derive only the equality interface actually used by the refutation compiler, exploiting a formula code as a fresh variable label. The diagonal version is already in the incoming PA research, and is credited as such. A small iPRA checker cannot simply replace the searched calculus: the PA translation uses induction for an arbitrary formula in finite bit comprehension. Section 5 explains this strength distinction; no identification of Con(iPRA) with Con(PA) is used [7].

3 Exact source and compilation

3.1 Source representation and invariant interfaces

The delivered `pa195/source.json` specifies every primitive source instruction, its binary address, and every ordinal successor. The instruction vocabulary is increment, conditional decrement, break, continue, and halt. Binary association determines how break and continue depths are resolved; equal source fragments can share physical reader states. Consequently equality of flattened opcodes alone would not be sufficient. The verifier compares the complete ordinal control-flow graphs with two independently structured resolvers.

The 3,700 sites consist of 1,468 increments, 1,115 conditional decrements, 1,116 jumps, and one halt. Thus the branch census is

$$1468 + 2(1115) + 1116 + 1 = 4815.$$

The longest path has 37 bits and the longest consecutive zero run is six. The clean seed remains source ordinal 25, at path `100000`.

At a clean physical frame the left marker is immediately left of the first path cell, a zero-filled gap separates the path from the right header, and the complete register file follows the header. A natural

value v is encoded by $0\,1^{v+1}$. Logical registers $0, \ldots, 8$ occupy physical positions

$$(6, 2, 0, 4, 3, 7, 8, 5, 1).$$

The symbolic gap is $48 + n$ for arbitrary $n \geq 0$. Increment preserves the gap and appends a one-valued spare block; a positive decrement increases the gap by one; zero decrement and jump preserve it. The proof covers every complete finite file of at least ten registers satisfying the indicated branch condition.

3.2 The emitted numerical calculus

Section 5 defines the formula constructors and the changed gate fully. For completeness, the six unchanged heads are listed here. D, A, H are arbitrary numerical parameters, read as variable labels or formula codes in the indicated positions.

Head	Formula
1	$(\exists x_D\, U(D, H)) \to H$
2	$U(D, U(A, H)) \to U(A, U(D, H))$
3	$M(H, D) \to (E(H, A) \to M(A, D))$
4	$E(H, D) \to (M(A, H) \to M(A, D))$
5	$\exists x_D\, E(H, D)$
6	$U(D, A) \to (U(D, A \to H) \to U(D, H))$

The restricted propositional schema is

$$\rho_8(D, A, H, u) = ((H \to u) \to D) \to ((D \to H) \to (A \to H)), \qquad u \leq D.$$

The only primitive inference rules are modus ponens and generalization of an already derived theorem. There is no extra equality rule or arbitrary-tautology oracle.

In a command store (t, w, d, h), an AX call with parameters (c, x, y) uses $D = d + x$, $H = h + y$, $A = c$. In execution order it pushes the saved t, $U(d, h)$, $\rho_8(D, A, H, d)$, and heads 1 through 6. Its new top is the diagonal gate when the guard succeeds and truth otherwise; d, h are cleared. The exact output, including its full stack order and the gate, is verified as an identity in a free algebra. It is not inferred from selected small numerical codes.

The new `grammar.json` defines the complete association of the same 191 macro tokens. A pair of adjacent increments uses an already available macro root, so the grammar's number of named rules must not be confused with its 68 newly allocated reader controls. The deterministic builder expands the exact grammar, retains the inherited macro entry controls, and then applies the seven folds.

3.3 Exact startup and total primitive simulation

After exactly 58,154 literal steps, the delivered machine is in dense state 75 (native 84), with head at absolute coordinate -2. Relative to that head the left marker is at -1, the path is `100000`, the right header is at 48, and the physical register file is

$$[0, 0, 0, 0, 0, 0, 2, 0, 0] \;+\!\!+\; [1]^{26}.$$

The full finite nonzero support, containing 66 cells, is recorded in `startup_certificate.json`; every other tape cell is zero. The proof therefore uses blank initialization of this exact machine, not an externally supplied nonblank input.

For both the unfolded and folded tables the executable symbolic interpreter checks all 4,815 branch contracts at arbitrary gap and register values. Its register oracle is justified by the inherited unary-loop proofs. Independently, all 44 rows named in the Lean `boxRows` contract, the additional increment-tail row, and even the full 46-row protected superset were compared literally. Every row is unchanged. The runtime soundness argument and its PA internalization are developed in Section 5.

4 The seven-state quotient: a global finite certificate

Let R be the explicitly supplied unfolded 202-state table and let M be `pa195.native.tm`. Both use native start label 74 and native halt label 254. The dense presentation of M has start 0, working labels $0, \ldots, 194$, and halt label 195. The quotient map σ identifies the following seven pairs and fixes all other controls. The first member of each pair is removed.

$$(2018, 2024),\ (2030, 2050),\ (2038, 2019),\ (2042, 2028),$$
$$(2053, 2043),\ (2055, 2034),\ (2062, 2036).$$

The unfolded machine consists of 134 inherited controls and 68 newly allocated AX reader controls. All seven identifications are disjoint, so $202 - 7 = 195$ working controls remain. The root-reachability census visits every one of the 195 controls; no distinguished halt control is counted.

The incoming-history invariant. For every pair (a, b) in this list, every incoming row of R whose target is a writes 0 and moves right; every incoming row whose target is b writes 1 and moves right. There are exactly 22 such incoming rows, all listed in `local_fold_certificate.json`. Since neither a nor b is the start control, every reachable configuration of R satisfies

$$q = a \implies t(h-1) = 0, \qquad q = b \implies t(h-1) = 1,$$

where h is the head position and t is the tape. This is a direct one-step induction. It applies to every initial tape, not only to the blank run or to well-formed register-program frames.

The 28 local obligations. For each of the 14 old controls and each current bit $v \in \{0, 1\}$, start M in control $\sigma(q)$ at head position 0, with the prescribed history bit at -1 and current bit v at 0. The certificate gives a positive finite run that reproduces the single R transition, including its write, head displacement, and projected successor. Until that endpoint every read and write is at -1 or 0. The two-cell tape at the endpoint is compared exactly. Hence every extension of those two cells to a bi-infinite tape satisfies the same identity. Of the 28 runs, ten have length 1, seventeen have length 3, and one has length 5. All other old transitions commute with σ in exactly one step.

Halting preservation in both directions. Map a configuration (q, h, t) to $(\sigma(q), h, t)$. The preceding finite identities and incoming-history invariant show that every step of R from a reachable configuration is simulated by one to five steps of M. Every intermediate control has an outgoing transition unless the final endpoint is the halt control; σ fixes that halt control. Concatenation therefore sends any R halting trace to an M halting trace.

Conversely, suppose M halts after T transitions. If R does not halt within its first T transitions, its first $T + 1$ steps exist. Their simulating blocks concatenate to more than T defined M transitions, in contradiction with determinism and the absence of outgoing rows at halt. Consequently R halts within T steps. This also covers halts inside an attempted comparison block; an intermediate premature halt would contradict its verified finite trace.

All quantified tape and trace operations in this proof have standard primitive-recursive finite encodings. The invariant is an induction on trace length, the local lookup consists of 28 fixed finite cases, the forward trace construction concatenates blocks of length at most 5, and the converse uses bounded iteration through $T + 1$ parent steps. Thus the argument formalizes in PA and proves

$$\mathrm{PA} \ \vdash \ \mathrm{Halt}(R, \mathrm{blank}) \ \leftrightarrow \ \mathrm{Halt}(M, \mathrm{blank}).$$

No assumption of consistency, standard-model soundness, or nonhalting enters this quotient theorem.

4.1 The interacting five-step case

Compatibility is substantive: an apparent seven-edge matching in an earlier grammar allowed only five folds simultaneously. That grammar was discarded. The retained grammar passes the simultaneous check, including a case whose helper itself participates in another fold.

For raw control 2036 reading 1 with its required left-neighbor bit 1, the raw step writes 1, moves right, and enters 2037. The folded table executes these five rows; head positions are relative to entry.

State	Head	Read	Write	Move	Next
2036	0	1	1	L	2046
2046	-1	1	1	R	2043
2043	0	1	1	L	2005
2005	-1	1	1	R	2054
2054	0	1	1	R	2037

Only the two known cells are read, their contents are unchanged, and the final state and head exactly match the raw step. This illustrates why disjoint pairs alone are insufficient: the composed helper behavior must also be checked.

5 The PA-provable consistency equivalence

Let M_* denote the delivered 195-state table, with its designated blank initial configuration, and let H_* be the arithmetic assertion that this literal machine has a finite halting trace. Throughout this section, Con(PA) means that a fixed standard, recursively checked first-order PA proof system has no proof of $0 = 1$. Changing between the usual PA proof codings gives primitive-recursive proof translations whose correctness PA proves. The conclusion concerns the arithmetic sentence for the exact table, rather than an external assertion about its standard execution.

Theorem 5.1. *For the table M_* in this package,*

$$\mathrm{PA} \vdash (\neg H_* \leftrightarrow \mathrm{Con(PA)}).$$

This is an ordinary mathematical proof of a PA-provability statement. The new endpoint has not been compiled as an additional Lean or Rocq theorem. The executable certificates supply the finite literal-program facts used in the proof; the arguments below explain their arithmetic meaning and the uniform proof transformations.

There are three levels to connect: PA proofs, the numerical proof calculus searched by the register program, and literal binary-tape traces. The published PA196 development supplies the PA-refutation compiler for the original calculus [1]. Its other internal direction is not assumed here. We first justify the incoming diagonal replacement used by M_* [4], and then supply the reverse proof transformation and the finite-trace argument, developing the syntactic route identified in the earlier bridge notes [3].

5.1 The numerical calculus and its diagonal replacement

Write

$$p(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \quad I(A,B) = 4p(A,B), \quad M(a,b) = 4p(a,b)+1,$$
$$U(a,B) = 4p(B,a)+2, \qquad E(a,b) = 4p(a,b)+3.$$

The notational convention is $\neg A = I(A,1)$, conjunction and existential quantification being their classical abbreviations. Code 0 is truth, and code $1 = M(0,0)$ is the designated contradiction. The quantifier constructor has the transposed Cantor arguments displayed above.

The diagonal calculus $\mathcal{C}_\Gamma$ retains the six PA196 heads, the dominated propositional head, modus ponens, and closed generalization. Its guarded gate is

$$\begin{aligned}
\Gamma(B) &= I(B, U(B,B)),\\
V(a,b,c) &= I(M(a,b), U(c, M(a,b))),\\
P(a,b,c) &= I(c, M(c,b)),\\
S(a,b,c) &= U\big(b, (M(a,b) \to P(a,b,c)) \wedge (M(c,b) \to c)\big),\\
Q(a,b,c) &= U\big(c, S(a,b,c) \to \neg P(a,b,c)\big),\\
F_\Gamma(a,b,c) &= \neg\big(\Gamma(c) \to (V(a,b,c) \to Q(a,b,c))\big).
\end{aligned}$$

The gate is available when $c \neq a, b$; a failed guard emits 0. The logical rules and remaining axiom families are exactly those of `PA196/Logic/Basic.lean`, with this one replacement of its `bundle`.

Every recursive formula child of a nonzero code is smaller than that code. Every variable label, including a bound label, occurring in a formula of code B is strictly below B. These assertions follow from the inequalities $a, b \leq p(a,b)$ and the four tags, with code zero handled separately. Consequently x_B is absent from the decoded formula B. The diagonal clause is a valid vacuity instance, although its binder is chosen from the numerical code of the entire formula.

Classical implicational reasoning extracts from the gate both $\Gamma(c)$ and the old gate $G(a,b,c) = \neg(V(a,b,c) \to Q(a,b,c))$. Every diagonal instance $\Gamma(B)$ is available: use $c = B$ and $a = b = B+1$. Thus all membership-vacuity instances and pointed separation used in the original proof remain available.

Lemma 5.2 (Recovery of the required equality interfaces)**.** *The diagonal calculus derives, uniformly in the numerical labels,*

$$\begin{aligned}
&E(a,b) \to \big(M(d,a) \leftrightarrow M(d,b)\big),\\
&E(a,b) \to \big(M(a,d) \leftrightarrow M(b,d)\big),\\
&E(a,b) \to R(a,b,z) \quad (z \neq a,b),
\end{aligned}$$

where

$$R(a,b,z) = U\big(z, M(a,z) \leftrightarrow M(b,z)\big) \wedge U\big(z, M(z,a) \leftrightarrow M(z,b)\big),$$

and it derives $\exists x_a R(a,b,z)$ for $z \neq a,b$. All these derivations are supplied by primitive-recursive proof templates, with correctness provable in PA.

Proof. The two forward membership rewrites are retained heads. To obtain the reverse directions, first assume $a \neq b$. Take $\chi(t) = M(d,t)$ or $M(t,d)$, initially with $d \neq a$, or take the diagonal family $\chi(t) = M(t,t)$. Put

$$C = E(a,b) \to (\chi(b) \to \chi(a)), \qquad \varphi = \neg C, \qquad c = \ulcorner \varphi \urcorner.$$

Here c is the actual numerical code of the displayed formula, so it is larger than every label in C. In particular, all required guards hold. Apply the old gate at (a, a, c) and abbreviate its S, P, Q by those letters. We need only its consequence $\neg Q$, not an unproved general rule for changing bound-variable names.

The second conjunct in S gives $S \to U(a, M(c, a) \to \varphi)$. Propositionally, φ implies $E(a, b)$, $\chi(b)$, and $\neg\chi(a)$; hence universal monotonicity gives

$$S \to U(a, M(c, a) \to \neg\chi(a)).$$

The equality atom has disappeared from this quantified consequent. The unchanged negative-property specialization lemma now gives

$$S \to (M(c, b) \to \neg\chi(b)). \tag{1}$$

For precision, this specialization uses only the retained witness $\exists x_a E(b, a)$, direct rewriting under $E(b, a)$, and vacuity of $M(c, b) \to \neg\chi(b)$ at a. The latter follows from membership vacuity because $a \neq b, c$ and, in the two nondiagonal families, $a \neq d$. It is exactly the construction `negativeSpecialization` in `PA196/Logic/Reverse.lean`; none of its premises is primitive equality vacuity.

Under φ, the point condition P gives $M(c, a)$. Direct element-slot rewriting under $E(a, b)$ then gives $M(c, b)$. Together with (1), S contradicts the consequence $\chi(b)$ of φ. Classical deduction gives $\varphi \to (S \to \neg P)$. Generalize this theorem at c, distribute the quantifier through implication, and use $\Gamma(\varphi) = \varphi \to U(c, \varphi)$. The result is $\varphi \to Q$. Since the gate supplies $\neg Q$, double-negation elimination gives C, as required.

If $a = b$, the reverse rewrite is propositional. When the fixed label $d = a$, direct rewriting first takes $M(a, b)$ or $M(b, a)$ to $M(b, b)$; the already proved diagonal case takes $M(b, b)$ to $M(a, a)$. This covers every label coincidence without a freshness assumption on the theorem being derived.

For the neighborhood formula choose $z_0 = \ulcorner E(a, b) \urcorner$. The available diagonal clause $E(a, b) \to U(z_0, E(a, b))$ and the two bidirectional rewrites yield $E(a, b) \to R(a, b, z_0)$. To change the binder to an arbitrary $z \neq a, b$, only the case $z \neq z_0$ remains. Let $P(t)$ be either membership-only matrix in R. Bidirectional rewriting gives $E(z, z_0) \to (P(z_0) \to P(z))$. The witness $\exists x_{z_0} E(z, z_0)$, universal distribution, and vacuity of $P(z)$ at z_0 give

$$U(z_0, P(z_0)) \to P(z).$$

Its antecedent is vacuous at z: its atoms contain only a, b, z_0, and membership vacuity extends through its Boolean connectives and the outer quantifier by the retained commutation head. Generalization and distribution therefore give $U(z_0, P(z_0)) \to U(z, P(z))$. This proves the claimed neighborhood implication for every allowed z. Finally the unchanged witness head and existential monotonicity give the neighborhood witness, reversing its two arguments by the propositional symmetry of R if necessary.

Every operation in this proof is a fixed syntactic proof template, composition, numerical code formation, or a displayed decidable label-equality case split. The choices $c = \ulcorner \neg C \urcorner$ and $z_0 = \ulcorner E(a, b) \urcorner$ use no unbounded search. PA verifies the templates by the same proof-checker induction as their original counterparts. This establishes the asserted uniformity. □

The PA196 refutation compiler uses primitive equality through these membership-rewriting, neighborhood, and witness interfaces. In its logical and arithmetic certificate copies, the uses of equality vacuity are in the reverse-rewriting and neighborhood modules just replaced. The old gate, all other heads, and the command stack interface remain available. Substituting Lemma 5.2's templates therefore gives a PA-verifiable refutation compiler for $\mathcal{C}_\Gamma$. In particular,

$$\mathrm{PA} \vdash \exists p\ \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner) \implies \exists d\ \mathrm{Prf}_\Gamma(d, 1). \tag{1}$$

This is reuse of the published proof-producing construction with explicit replacement templates, rather than an appeal to standard-model equiconsistency. The 191-item AX word still pushes nine items in the same positions as the original nine-item command, so its stack compiler indices need no alteration.

5.2 A uniform translation of the numerical calculus into PA

We now prove the converse proof-theoretic direction. Work first in the fixed conservative definitional extension of PA containing $\mathrm{pow2}(n) = 2^n$ and its usual arithmetical definition. Only fixed primitive-recursive functions are added. A fixed primitive-recursive elimination of these definitions converts the resulting proofs to ordinary PA proofs.

Use the bit predicate

$$B(s,z) \;\equiv\; \exists u, v\,[s = (2u+1)2^z + v \;\wedge\; v < 2^z].$$

PA proves the usual digit lemmas directly by induction and division with remainder. In particular $B(s,z) \to z < s$, since $s \geq 2^z \geq z+1$, and hence PA proves $\neg B(x,x)$.

Let $T(c)$ be the arithmetic formula obtained by decoding the source code c:

$$\begin{aligned} T(0) &= (0=0), & T(M(a,b)) &= B(x_a, x_b),\\ T(E(a,b)) &= (x_a = x_b), & T(U(a,A)) &= \forall x_a\, T(A). \end{aligned}$$

For a nonzero implication code the translation is the corresponding implication. There is one numerical alias: $I(0,0) = 0$. Thus the uniform constructor theorem is a proof of

$$T(I(A,B)) \leftrightarrow (T(A) \to T(B)),$$

not an unconditional assertion of syntactic identity. At $A = B = 0$ it uses the fixed proof $(0=0) \leftrightarrow ((0=0) \to (0=0))$. Every other case follows from the definition of T.

Arithmetic auxiliary bound variables are represented with de Bruijn indices, or equivalently renamed above all source labels before substitution. The source label c is therefore absent among the free variables of $T(c)$ even though the expanded bit formula has its own bound variables. Quantifier formation closes the indicated free source label, so nested occurrences of the same label have their usual shadowing semantics. No assumption that all numerical labels are distinct is made.

Lemma 5.3 (Uniform finite bit comprehension)**.** *Given the syntax code of any arithmetic formula $\theta(z, \vec{u})$, there is a primitive-recursive procedure producing a PA proof of*

$$\forall n\, \exists s < 2^n\, \forall z < n\, \big(B(s,z) \leftrightarrow \theta(z,\vec{u})\big). \tag{2}$$

PA proves that the procedure produces a valid proof for every well-formed formula input, with the indicated fresh variables.

Proof. Take induction on n with the existential formula in (2) as induction formula. At zero choose $s = 0$. Given a witness $s < 2^n$, split by excluded middle on $\theta(n, \vec{u})$. If it holds use $s + 2^n$; otherwise use s. The fixed bit lemmas say that the first n bits are unchanged, that the new nth bit is respectively one or zero, and that the result is below 2^{n+1}. This is a fixed finite first-order proof pattern with one PA induction instance.

On syntax codes the procedure performs capture-avoiding substitution, forms the displayed induction instance, and concatenates this fixed proof pattern with the fixed arithmetic lemma proofs. It never evaluates θ. Its syntax operations and a fresh-bound-variable choice are primitive recursive. The PA proof of correctness is induction on the syntax operations, followed by direct verification that the

generated induction line is an instance of the PA induction schema and that each subsequent line follows by the stated logical rule. Thus this is a statement about proof checking, with an arbitrary formula code as input, not a uniform satisfaction or reflection principle for PA. Full PA induction is used: θ is not restricted to a bounded or quantifier-free formula. □

We spell out the gate supplier because label collisions matter here. Fix a, b, c with $c \neq a, b$, let $\phi = T(c)$, and regard x_a, x_b as the free values before the inner quantifier is entered. If $a \neq b$, apply (2) with bound $x_a + x_b + 1$ and matrix

$$\big(B(x_a, z) \vee z = x_b\big) \wedge \phi[z/x_b]. \tag{3}$$

The witness s satisfies, for every z,

$$B(x_a, z) \to (\phi[z/x_b] \to B(s, z)), \qquad B(s, z) \to \phi[z/x_b],$$

and $\phi \to B(s, x_b)$. To pass from the bounded comprehension formula to the displayed unbounded assertions, use $B(x_a, z) \to z < x_a$ and the fact that a number below 2^n has no set bit at an index at least n.

If the *labels* $a = b$ coincide, the first antecedent inside $T(S)$ becomes $B(z, z)$, not $B(x_a, z)$ with x_a held fixed. It is therefore false. Apply (2) instead with bound $x_b + 1$ and matrix

$$(z = x_b) \wedge \phi[z/x_b]. \tag{4}$$

This witness has the required second implication and distinguished point condition, and the first implication is vacuous. Equality substitution in (4) supplies ϕ at the distinguished point. The substitutions in (3)–(4) are capture avoiding; in particular the inner binder corresponding to b is renamed when the witness expression depends on the original free value of x_b.

In both cases the result is a PA proof of $\exists x_c\,(T(S) \wedge T(P))$. Freshness of c in ϕ ensures that choosing the set witness does not alter the property. Consequently PA proves $\neg T(Q)$. The guarded membership-vacuity clause has a first-order proof, and $T(\Gamma(c))$ has a first-order proof by the code-freshness lemma. Classical propositional reasoning therefore proves $T(F_\Gamma(a, b, c))$. When the guard fails the command emits 0, with its fixed truth proof.

The source false code is translated to $T(1) = B(x_0, x_0)$. Its negation is a fixed PA theorem, uniformly under all binders. Thus source negation, conjunction, and existential abbreviations have PA-provable equivalences to the ordinary classical connectives, even if a surrounding source quantifier binds label zero. This deals with the false-code convention independently of the truth-code alias.

The other head suppliers are fixed first-order proof templates: the witness-elimination head, quantifier commutation, the two equality congruence heads, an equality witness, and quantifier distribution. Coincident labels are handled by capture-avoiding instantiation, with a numerical equality case split if needed. The dominated propositional head is a classical tautology; its arithmetic validity does not require the dominance restriction. Generalization and modus ponens use their ordinary PA proof constructors, together with the implication-constructor equivalence above.

Lemma 5.4 (Syntactic refutation translation)**.** *There is a primitive-recursive function J such that*

$$\mathrm{PA} \vdash \forall d\ (\mathrm{Prf}_\Gamma(d, 1) \to \mathrm{Prf}_{\mathrm{PA}}(J(d), \ulcorner 0 = 1 \urcorner))\,. \tag{5}$$

Proof. Decode the source derivation by recursion on its finite proof code. At an axiom node insert the appropriate supplier just described. At modus ponens concatenate the two translated proofs and the compositional implication proof; at generalization append the corresponding quantifier rule. At a code-zero alias emit or use the fixed truth equivalence. Recursion on a malformed proof is made total by an arbitrary fixed output; the correctness statement has the displayed proof-validity hypothesis.

PA proves the syntactic invariant $\mathrm{Prf}_\Gamma(d, A) \to \mathrm{Prf}_{\mathrm{PA}}(J_0(d), T(A))$ by induction on this coded recursion. Constructor children decrease strictly; proof concatenation, label closing and fresh-variable renaming have primitive-recursive definitions and their usual syntactic correctness inductions. The gate case is precisely Lemma 5.3 and the two label cases above. Append the fixed theorem $\neg B(x_0, x_0)$ at a proof of code one, then eliminate the fixed definitional extension. This gives (5).

This argument proves one quantified assertion about proof codes inside PA. It does not merely produce a PA proof separately for each standard formula and then assert a uniform conclusion. The inductive verification of the compiler, including the schema supplier, is the step that makes (5) applicable to nonstandard codes in every model of PA. □

5.3 The exact proof-store invariant

The numerical command store is (t, w, d, h). Both t and h carry formula codes; w is the Cantor-coded stack and d is an arithmetic parameter. Annotate t, h with PA proofs of $T(t), T(h)$, and annotate each live stack element with the corresponding proof. The clean store uses the fixed proof of $T(0)$.

Here is a precise convention that avoids treating Cantor pairing as a free list constructor. Let hd, tl be the two Cantor projections. At zero both are zero. For $w > 0$, $\mathrm{tl}(w) < w$: writing $w = p(a, b)$, if $b > 0$ then $p(a, b) > b$, and if $b = 0$ the inequality is immediate. Define

$$\delta(w) = \min\{k \le w : \mathrm{tl}^k(w) = 0\}.$$

PA proves the set nonempty by induction on w. The canonical stack has exactly these $\delta(w)$ heads, and its proof annotation is a sequence of that length. Its invariant is

$$\forall i < \delta(w) \quad \mathrm{Prf}_{\mathrm{PA}}(\pi_i, T(\mathrm{hd}(\mathrm{tl}^i(w)))). \tag{6}$$

Bounded minimization and bounded sequence operations make this an ordinary primitive-recursive invariant.

Pushing a code a onto w prepends its proof, except when $a = w = 0$; in that one case $p(0, 0) = 0$ and the canonical annotation remains empty. Popping nonzero w removes the first proof. Popping zero returns code zero with its fixed truth proof and leaves an empty annotation. These statements are exact consequences of $\mathrm{decode}(p(a, b)) = (a, b)$ and $p(a, b) = 0 \leftrightarrow a = b = 0$, and PA proves each of them.

Now inspect the actual command updates. AX pushes the saved top, a generalization of the old h, and the seven ordinary head instances, in their specified order, and leaves the guarded diagonal gate in t with $d = h = 0$. The nine pushed entries have the PA proofs provided above. For MP with offset x, put $y = d + x$. If $I(t, y) = h$, combine the proofs for t, h to obtain a proof for y, push the truth code, and set $(t, d, h) = (y, y, 0)$. If $I(t, y) = h = 0$, then $t = y = 0$ and the same operation is justified by the truth proof. On a failed match the command outputs truth, retains the stack, and sets $(t, d, h) = (0, y, 0)$. Shift, when it does not halt, sets

$$(t, w, d, h) \longmapsto (h, \mathrm{tl}(w), d, \mathrm{hd}(w)),$$

and moves the annotations accordingly.

The literal Shift test is $t + h = 1$. PA proves that this implies $t = 1$ or $h = 1$. In either case the relevant annotation, followed by the fixed proof $\neg T(1)$, yields a PA refutation. In particular the invariant must track both visible formula registers, rather than just t.

Every annotation update is primitive recursive. The local statement just proved holds for *every annotated store*: a successful command preserves the annotation invariant, and a halting command produces a PA refutation. PA induction therefore propagates the invariant through an arbitrary coded

finite command sequence, including one starting at an already live scheduler store. In particular, if execution from the clean store halts then a PA refutation exists. Under Con(PA), every annotated command has a nonhalting output and its successor store is annotated again. This is the internal replacement for the upstream external HF-model soundness argument.

5.4 Why the finite runtime certificates are sufficient in PA

The source and physical verifications in this package have the following mathematical content. A frame consists of its source path, a gap, and the complete register file. A value v is stored as $0\,1^{v+1}$. The physical gap is represented by an affine run length $n + c$, with arbitrary $n \geq 0$; register contents and the complete tail are left symbolic. The initial file has at least ten blocks, and this lower bound is preserved by every primitive: increment appends one extra block and decrement retains the length. The greatest source path length is 37, below the header constant 48.

For each increment, each jump, each zero and positive decrement branch, and the halt site, the delivered finite trace certificate starts at the exact frame and ends at the exact successor frame or at the distinguished halt control. There are 4,815 cases for the 3,700-site source. These are symbolic branch contracts, rather than 4,815 examples of numerical register inputs.

There are only four kinds of step in their symbolic interpreter: an actual table row; a change of notation for an identical tape; a whole run of one self-loop; and a register-file excursion. Notation changes preserve every concrete instance. A self-loop over a run of length $an + c$ has the stated effect by induction on that length. The register excursions are marking a register, incrementing it, zero decrement, and positive decrement. Their proofs in `M6/Native.lean` are inductions on the finite file and its unary blocks. They use exactly the specified engine rows and the append-one tail row. These rows are unchanged by the new decoder and the seven simultaneous folds.

Consequently the soundness proof of `M6/Runtime.lean`, specifically `sstep_sound` and `exec_sound`, applies to each case. It gives exact finite reachability for every natural gap and every valid full register file. The checker only accepts after it has traversed at least one actual row, so each completed primitive segment has strictly positive physical length. A nonhalting contract ends at the root control, not at the halt control.

The same assertion is provable arithmetically, for the following specific reason. Finite tapes are coded finite words; runs, concatenation, frame construction, and the register operations are primitive recursive. The proofs for a self-loop and each of the four register excursions use induction on a natural run length or a coded finite list. Their witness traces can be produced by the same bounded scans and primitive recursions. Repeating the four symbolic-step cases and concatenating their traces is again a primitive recursion. PA proves these inductions directly. The candidate-specific part is a finite list of table equalities and completed symbolic traces, all decidable closed arithmetic facts. Thus PA proves the exact, positive source-to-physical segment assertion uniformly in coded frames. An external theorem about standard tapes alone would not suffice; the just-described coded induction is the internalization used here.

The AX source binding checks every primitive and relative jump in each of its 191 macro blocks against the inherited total arithmetic macro, and checks every exterior successor. The free-algebra emitter check supplies its exact numerical output formulas. Hence the source-level command and scheduler contracts are inherited with the gate replaced as stated. The scheduler's other 169 sites are unchanged. Its finite command-word decoding, the Cantor-pairing routines, and the fixed AX macro composition are primitive-recursive computations with PA-provable terminating bounds. More specifically, the construction `parsed_command_step` in `PA196/Source/CommandTransfer.lean` takes the next decoded command at a fetch boundary, executes it, and returns to the next fetch boundary in

a strictly positive number of primitive source steps whenever the decoded command has a nonhalting output. It applies to every store and every scheduler pair (n, l) satisfying that explicit output premise; the annotated-store invariant under consistency supplies the premise used below. The source binding retains this construction with the new AX composition. The store need not be reset between decoded words; the proof-store invariant is preserved from one command boundary to the next.

The complete blank startup is checked for exactly 58,154 steps, including the entire final nonzero support, the head position, and the state. It ends at the stated clean source frame. In native labels that state is 84 and the head coordinate is -2; the injective dense relabeling is checked separately. As a fixed finite computation, the startup equality and the absence of an earlier halt are PA-provable.

Lemma 5.5 (Reflection of a finite physical halt)**.** *PA proves that a finite halting trace of M_* yields a PA refutation.*

Proof. Work in PA and assume Con(PA). At a command boundary, decode the next command by the fixed primitive-recursive function parse(seed(n, l)). The proof-store invariant makes its Shift halt test impossible, and its nonhalting update supplies the next annotated store. The positive `parsed_command_step` contract just described then supplies a finite source execution to the next command boundary. This is exactly the strengthened syntactic version of the `SoundBoundary/boundary_progress` argument in `PA196/Source/Nonhalting.lean`; the external appeal to an HF model is replaced by the PA proof annotations and consistency.

For any given T, primitive recursion through $T + 1$ such command boundaries therefore constructs more than T primitive source steps from the clean initial source frame. Every command segment has positive length and ends at the next nonhalt fetch boundary. The intermediate macro routines terminate by their stated arithmetic contracts. Thus no source halt can occur inside an unfinished final macro either. Concatenating the exact positive physical segments for these primitive steps constructs a physical run from the same initial frame longer than T.

Now suppose there were a blank-tape halting trace of length T. If it stops within startup, the fixed startup computation refutes it. Otherwise concatenate startup and the physical segments just constructed. Their total length exceeds T. Determinism forces the alleged halted configuration to be the same configuration at that index in the constructed run. With the partial operational semantics there is no outgoing row at halt, a contradiction. With an absorbing totalization of halt the argument is equivalent: the run could never again reach a nonhalt root frame.

No inverse decoding of an arbitrary intermediate tape is used. Nor do we need an unbounded least-time search for the next frame. The chosen number $T + 1$ of completed command segments is a bounded coded iteration. PA proves sequence concatenation and the determinism comparison by induction on finite trace length. We have therefore proved in PA that consistency excludes H_*, or equivalently $H_* \to \neg\,\mathrm{Con(PA)}$. □

For clarity, exact reachability to a nonhalt endpoint already rules out a premature halt inside a primitive segment: a deterministic partial run cannot pass through a state with no outgoing row. The required runtime premise is therefore the exact positive endpoint contract verified here, not a disjunction allowing an unexplained early halt instead of the next source frame.

5.5 Completing the other direction

By (1), a PA refutation produces a $\mathcal{C}_\Gamma$ refutation. The unchanged nine-item command compiler turns that derivation into a finite command word which halts, using the new gate supplier where required. Here one uses the existing proof compiler's protected-stack-tail and scheduling identities, which permit

the actual live store; no reset between candidate words is being assumed. The unchanged scheduler reaches the compiled word, or halts earlier, by the same bounded candidate induction as in the original compiler. The total arithmetic macro implementations and the source-to-physical contracts then give a finite blank-tape halt of the exact delivered table. These are proof-producing finite computations and their correctness inductions take place in PA. Therefore

$$\mathrm{PA} \vdash \neg\,\mathrm{Con}(\mathrm{PA}) \to H_*.$$

Combining this with Lemma 5.5 and classical contraposition proves Theorem 5.1.

5.6 Logical strength and the role of ConciseIPRA

The proof does not replace PA by intuitionistic primitive recursive arithmetic, the subject of the ConciseIPRA project [7]. The finite-comprehension supplier uses an induction instance for an arbitrary formula, which is precisely why a short iPRA proof checker cannot simply be substituted as the searched theory. The transferable ideas are smaller derivable interfaces, freshness read from syntax, and uniform proof-certificate manipulation. Here the diagonal code-freshness device is already present in the incoming PA research [4]; the new reduction is the compatible physical allocation of that source, not a new claim that iPRA has PA's consistency strength.

6 Verification, reproduction, and evidence limits

6.1 Reproduction without a search

From the archive's top-level directory, run

```
python3 -B pa195/build_verify.py
python3 -B verification/verify_independent.py pa195 --concrete
python3 -B verification/check_engine.py
```

Only Python 3 and its standard library are required. No network access, optimizer, SAT solver, or proof-assistant installation is needed for these checks. Python optimization mode `-O` must not be enabled, because the preserved source parser uses assertions. To regenerate derived certificates, add `-export` to the first command.

The builder pins the inherited files, reconstructs both tables, checks the complete state renaming, validates all incoming-history conditions and local quotient traces, verifies the source graph and free-algebra emitter identity, checks both universal symbolic contract lists, and replays the complete startup. The independent verifier imports none of the compiler or symbolic-runtime code. It parses the source itself, resolves control through subtree intervals, checks the literal machine and conjugacy, and performs two independent startup simulations.

Evidence	Result
Literal table reconstruction	195 states, 390 rows; exact SHA-256
Graph reachability	All 195 working controls
Local quotient	22 incoming rows; 28 exhaustive two-cell cases
Positive local durations	Ten length-1, seventeen length-3, one length-5
Source identity	3,700 opcodes and all ordinal successors unchanged
Emitter identity	All 191 tokens; full stack and guarded top
Symbolic primitive verification	All 4,815 cases, for each of two tables
Register oracle prerequisites	All 46 protected rows identical
Blank startup	58,154 steps; state, head, and complete tape exact
Independent concrete frames	19,260 runs; 8,234,830 literal steps

The concrete frames use gaps 48, 49, and 61, variable register contents, both decrement branches, and different nonempty spare tails. Their longest measured run is 708 transitions. These are transcription and implementation controls; the universal proof is supplied by the symbolic contracts and their loop lemmas. Similarly, the symbolic evaluator's maximum of 497 is an *abstract* execution length, not a bound on the physical time for arbitrary unary register contents.

6.2 What the packet establishes

The seven-state quotient is supported by a complete finite all-tape proof. The source compilation is supported by exact graph identity, total symbolic contracts, and the inherited register-block proofs. The PA consistency calibration is given as an ordinary proof with uniform proof-code transformations, including the truth-code alias, the zero-stack alias, variable-label collisions, and both directions of finite trace transport.

No finite prefix of the actual blank run is used as evidence of nonhalting. The runtime and startup verifications do not themselves prove PA's consistency. The theorem only transfers consistency to nonhalting inside PA; it does not assert that PA proves either of those equivalent statements. No global minimality claim is made for 195 states.

The remaining difference from a fully formalized result is explicit: the new complete theorem has not been elaborated in Lean or Rocq. Selected unmodified upstream proof sources are supplied for review, with exact pinned URLs and hashes, rather than presented as a new standalone proof-assistant project.

6.3 Files and provenance

`pa195/` is the deterministic machine packet. `verification/` contains the independent checker and audit receipts. `comparison/` contains the complete archive inventory, the direct PA directory inventory, canonical comparison manifests, and the 25 normalized supported 196-state tables. `reference_sources/` supplies the selected upstream proof modules cited in the arithmetic and engine arguments. The original licenses and source hashes are retained.

The article is compiled with

```
cd report
latexmk -pdf -interaction=nonstopmode -halt-on-error pa195_conpa.tex
```

All numerical table data printed below are generated from the delivered machine, so the PDF and the machine-readable artifact specify the same 390 transitions.

A Complete transition table

An entry wDq' means write w, move D, and enter q'. State 0 is initial. The symbol H denotes the separate halt label 195. The 195 working-state entries below specify both transitions. Read down each panel, then proceed to the next panel on its right.

State	Read 0	Read 1	State	Read 0	Read 1	State	Read 0	Read 1	State	Read 0	Read 1
0	1 R 1	1 R 61	49	0 R 16	1 R 49	98	0 R 91	1 R 107	147	0 R 148	1 R 162
1	1 R 82	1 R 1	50	0 R 63	1 R 64	99	0 L 131	0 L 85	148	0 R 78	1 R 149
2	0 L 2	1 L 3	51	0 L 39	1 L 51	100	0 R 108	1 R 94	149	0 R 150	1 R 157
3	0 L 4	1 L 3	52	0 R 22	1 R 52	101	0 L 109	0 L 2	150	0 R 126	1 R 151
4	0 L 6	1 L 3	53	0 L 11	0 R 49	102	0 R 38	1 R 68	151	0 R 157	1 R 153
5	1 R 7	0 L 5	54	0 R 58	0 L 16	103	0 R 59	1 R 110	152	0 L 165	1 R 78
6	0 L 9	1 L 3	55	0 R 65	1 R 66	104	0 R 59	1 R 111	153	0 R 154	1 R 155
7	0 R 7	0 R 10	56	0 R 59	1 R 67	105	0 R 66	0 L 16	154	0 R 116	1 R 152
8	0 R 14	1 R 13	57	0 R 52	1 R 57	106	0 R 14	1 R 110	155	0 R 156	1 R 136
9	0 L 15	1 L 3	58	0 R 48	0 R 25	107	0 R 21	1 R 112	156	0 R 78	1 R 78
10	0 L 16	0 R 10	59	0 L 28	1 L 68	108	0 R 113	1 L 20	157	0 L 144	1 R 152
11	0 L 26	1 R 18	60	0 L 102	1 L 30	109	0 R 30	1 R 65	158	0 R 159	1 R 160
12	0 R 19	1 R 20	61	0 R 0	1 R 61	110	0 R 38	1 L 108	159	0 R 160	1 R 137
13	0 L 79	1 L 31	62	1 R 125	1 L 51	111	0 R 121	0 L 16	160	0 R 19	1 R 161
14	0 L 11	0 R 22	63	0 R 70	1 R H	112	0 R 114	0 L 16	161	0 R 58	1 L 191
15	0 L 23	1 L 3	64	0 L 126	1 R 31	113	0 R 24	0 R 47	162	0 R 180	1 R 166
16	1 L 2	0 L 2	65	0 R 40	1 R 60	114	0 R 48	0 R 32	163	0 R 12	1 R 164
17	0 R 24	0 R 32	66	0 L 79	1 R 71	115	0 R 70	1 R 86	164	0 R 116	1 R 138
18	0 R 84	0 L 95	67	0 R 53	1 L 72	116	0 R 19	1 R 118	165	0 R 137	1 R 136
19	0 R 28	1 R 29	68	0 R 73	1 R 74	117	0 R 126	1 R 28	166	0 L 190	1 L 177
20	0 R 30	1 R 31	69	0 R 69	1 R 75	118	0 R 122	1 R 107	167	0 R 164	1 R 78
21	0 R 24	0 R 57	70	0 R 76	1 R 77	119	0 R 94	1 R 86	168	0 L 155	1 L 176
22	0 R 33	1 R 22	71	0 R 79	1 L 110	120	0 R 83	1 R 98	169	0 R 152	1 R 58
23	0 L 34	1 L 3	72	0 L 98	1 R 54	121	0 R 87	1 R 87	170	0 R 171	1 R 175
24	0 L 35	0 R 1	73	0 R 48	0 R 33	122	0 R 17	1 L 109	171	0 R 163	1 R 172
25	0 R 36	1 R 25	74	0 R 81	0 L 41	123	0 R 70	1 R 101	172	0 R 161	1 R 174
26	0 R 122	1 R 41	75	0 R 50	1 R 132	124	0 R 96	1 R 108	173	0 R 116	1 L 139
27	0 R 38	0 L 39	76	0 R 130	1 R 21	125	1 L 39	1 L 99	174	0 R 155	1 R 58
28	0 R 40	1 R 41	77	0 R 101	1 R 90	126	0 R 80	1 R 78	175	0 R 154	1 R 172
29	0 R 42	1 R 43	78	0 R 92	1 R 93	127	0 R 120	1 R 133	176	0 R 158	1 R 173
30	0 R 44	1 R 45	79	0 R 73	1 R 72	128	0 R 124	1 R 123	177	0 R 178	1 R 182
31	0 R 21	1 R 46	80	0 R 91	1 R 119	129	0 R 123	1 R 128	178	0 R 179	1 R 181
32	0 R 47	1 R 32	81	0 R 95	1 R 96	130	0 R 44	1 R 90	179	0 R 58	1 R 180
33	0 R 32	1 R 33	82	0 R 62	0 R 1	131	0 L 5	0 L 131	180	0 L 170	1 R 165
34	0 L 69	1 L 3	83	0 R 27	1 R 97	132	0 R 127	1 R 59	181	0 R 156	1 R 116
35	1 R 35	1 L 2	84	0 R 89	1 R 88	133	0 R 11	1 R 37	182	0 R 173	1 R 168
36	0 R 49	1 R 36	85	1 L 99	1 L 85	134	0 R 135	1 R 185	183	0 R 166	1 R 169
37	0 R 28	1 R 20	86	0 R 19	1 R 100	135	0 R 136	1 R 139	184	0 R 116	1 R 173
38	0 L 11	0 R 36	87	0 R 58	1 R 58	136	0 R 12	1 R 137	185	0 R 186	1 R 194
39	0 L 5	1 L 51	88	0 R 117	1 R 115	137	0 R 86	1 R 138	186	0 R 187	1 R 190
40	0 R 24	0 R 25	89	0 L 26	1 R 134	138	0 R 78	1 R 87	187	0 R 116	1 R 188
41	0 R 13	0 L 16	90	0 R 95	0 L 131	139	0 R 140	1 R 183	188	0 R 189	1 R 159
42	0 R 53	1 R 54	91	0 R 102	1 R 103	140	0 R 141	1 R 166	189	0 R 168	1 R 78
43	0 R 55	1 R 56	92	0 R 67	1 R 56	141	0 R 64	1 R 142	190	0 R 191	1 R 192
44	0 R 24	0 R 52	93	0 R 55	1 R 104	142	0 R 143	1 R 177	191	0 R 184	1 R 189
45	0 R 58	1 R 59	94	0 R 21	1 R 105	143	0 R 12	1 R 144	192	0 R 167	1 R 193
46	0 R 60	0 L 16	95	0 R 21	0 L 10	144	0 R 145	1 R 176	193	0 R 168	1 R 122
47	0 R 25	1 R 47	96	0 R 59	1 R 41	145	0 R 86	1 R 146	194	0 R 129	0 L 131
48	0 L 35	0 R 61	97	0 R 106	1 R 8	146	0 R 147	1 R 170			

B Archive-by-archive comparison

Every archive below is at the pinned source commit. The longer machine-readable inventories preserve exact member names, sizes, digests, and comparison maps. The entries distinguish mathematical source simplification from a validated physical state reduction.

Archive	Contribution and status
`U208_ConPA_2026-10-29.zip`	Complete U208, original class A: identify old native roles 172 and 211, with helper 217 (or 214 under alternate native labeling). The local simulation is a finite all-tape argument; full PA calibration is inherited and written, not newly kernel exported.
`U208_ConPA_Conditional_Context_2026-10-29.zip`	Same conditional-history reduction; primary and alternate presentations give the two original U 208 graph classes. Four local blocks have lengths 7,1,3,1.
`U208_ConPA_Entry_Context_2026-10-29.zip`	Re-expresses the same reduction by entry guards, with primary and compatibility alternatives. Its saving is not additive with the previous packets.
`U208_ConPA_History_2026-10-29.zip`	Historical-label presentation of the same two classes and same one-state deletion; includes a failed further reduction.
`u208_research_2026-10-29.zip`	Both original U 208 classes with symbolic runtime and startup evidence; no further state saving.
`pa208_research_2026-10-29.zip`	Eighteen additional pairwise nonisomorphic U 208 helper variants, each with a local halting-equivalence argument. Expands accepted U 208 corpus to 20 classes; no accepted 207.
`Below_U208_Research_2026-10-29.zip`	Source-boundary guards, displaced-head quotient lemma, closed-loop pruning, and broader negative local searches. Pruning 3909 to 3904 source occurrences increases the shared-tree cost. No accepted 207.
`ConPA_below208_research_2026-10-29.zip`	Two-object obstruction to collapsing source/output cover roles, exhaustive restricted axiom-parameter permutation search, two-row and register-invariant searches. No accepted 207.
`pa_consistency_research_2026-10-29.zip`	Proves zero-guarded source specialization 3909 to 3904 instructions and preserves the ordinal control graph; no smaller physical table.
`u208_comparison_research_2026-10-29.zip`	Exact comparison, source/encoding research and a fully specified rejected 207 frame witness. Weak/exact schema equivalence is separate from the still-open runtime/calibration replacement.
`pa_unary_power_cover.zip`	A unary-matrix power-cover theory and written mutual consistency calibration with full PA. Supplies numerical proof-search interface and logical separations; no new literal TM.
`pa_tm_research_2026-11-13.zip`	New alternative 196 with head order (1,2,3,5,6,4), unchanged 3812-site CFG, maximum source depth 36 rather than 38. Physical Lean baseline rebuild is supplied. R6-pruned 198 passes runtime but lacks calibrated blank startup.
`pa_tm_extended_research_2026-11-13.zip`	Four distinct one-row closing-CD variants remain196, with constrained symbolic runtime and unchanged blank startup. Joint source invariants and a generalized register executor strengthen search.
`conpa_tm_comparison_and_search.zip`	Accepted AD-register-swap196. Two-rotation 195 is rejected by 93 primitive-contract failures. Source geometry changes cannot reuse old contextual folds automatically.
`pa196_research_2026-11-13.zip`	Exact 157-node representation of the original 3812-site source, down from159; seed stays at source path100000. Claimed prospective 195 allocation loses a required decoder helper. Native HA/full-induction theory is a separate research direction.

Archive	Contribution and status
pa_consistency_research.zip	New 158-node source and an accepted 196 regrouping; the eighth scanner zero consumes the apparent one-state saving. Its seven-zero 195 version misroots on a concrete source frame.
pa_consistency_reduction.zip	Rebracketed196; H-only Shift 196 with 3811 sites and 57922-step startup; integrated witness gate has 3745 sites but best physical table203. Two optimistic 195 allocations fail, one by an exact short cycle on a source frame.
pa_tm_comparison_and_reductions.zip	Two accepted 196 rotations. Equivalent gate reordering shortens AX 201 to 199 macro items and source 3812 to3773, but best physical compilation198. Finite free-algebra/logic certificates and restricted grammar searches are retained.
pa_consistency_research_2026-11-13.zip	Diagonal vacuity B→U(B,B), exploiting formula-code freshness; AX 201 to191, source 3812 to3700. Initial complete implementation199. Gives detailed written reverse PA proof translation; no new full Lean endpoint.
pa_consistency_research_2026-11-13(2).zip	New twelve-rotation 196 and diagonal198. Also a command calculus halting at successful MP deriving code1; no new binary table for that command change.
conpa_tm_comparison_and_reduction_search.zip	Two experimental 196 MP commands (result in H, omit zero push), with mathematical command arguments but only bounded physical frame evidence. Two rejected Shift layouts actually halt from blank after 43563 steps.
conpa_tm_study_2026-11-13.zip	Combined 196 implements H-only Shift and MP-in-H, with 4928 uniform contracts; an additional rebracketed Combined 196 has full runtime and startup confirmed in this audit; direct-negation 197 has 3556 sites. A 157-node source receives a larger 207-state realization.
pa_consistency_comparison_2026_11_13.zip	First quantifier head split into modal B plus explosion, with finite Hilbert certificates in both directions. Countermodels obstruct deleting either membership rewrite head or replacing the first quantifier head by reflexivity. Twelve source/table combinations retain separate command/calibration obligations.
pa_consistency_comparison_and_new196_2026-11-13.zip	Three accepted diagonal-calculus 196 layouts: diagonal 196 (191 AX items,3700 sites), separate 196 (190 items,3673 sites), scanner_best 196 (same 3673 sites, improved address layout). Also a runtime/startup-checked scanner 196 on original source and four auxiliary diagonal closing-CD variants with differential runtime proofs, trit invariants, and exact startup. Short-scanner 195 and U208/PA 196 hybrid startup attempts fail stated obligations.
pa_below196_research.zip	Eight equivalent gate spellings, orientation involutions, exact freshness obstruction R(5)=9, smaller pairing macro that increases DAG cost, H-only guard specialization, and broader finite physical searches. No new accepted smaller table.
pa_tm_research_2026-11-13(2).zip	New zerojump 196 and an auxiliary CD3 parent (fully replayed in this audit): native27/1 changes 0 L40 to1 R21 under the source26 R6=0 invariant; startup58402. Deleting the resulting prospective helper gives a 195 table with wrong source successor and R6 increment, witnessed also on the actual blank execution.
PA_consistency_TM_research_2026-11-13.zip	New source-only zero-run-five witness (3812 sites,159 internal nodes,depth37), combined-negation/MP 3573-site 157-node witness, and parameter-free residual MP. Fixed/protected physical allocations remain rejected; many bounded quotient searches are negative.

References

[1] James Chang. *Nonhalting and PA-independence for a 196-state Turing machine*. Upstream revision 407cdc356f6b586635e71ac92be394255f4bfea4. https://github.com/jamesyc/BusyBeaver/tree/407cdc356f6b586635e71ac92be394255f4bfea4. The imported paper, source, machine, and proof inventory are in ProveIt, Computability/BusyBeaver/Research/PA/jamesyc-upstream/pa196/.

[2] ProveIt. *PA196 and the incoming U208 construction*. Commit 906367250a7f96c923147b10df392294bced7be5. https://github.com/VladimirReshetnikov/ProveIt/blob/906367250a7f96c923147b10df392294bced7be5/Computability/BusyBeaver/Research/PA/pa196-vs-u208/COMPARISON.md.

[3] ProveIt. *PA196: a route to the missing internal implication*. Same pinned commit. https://github.com/VladimirReshetnikov/ProveIt/blob/906367250a7f96c923147b10df392294bced7be5/Computability/BusyBeaver/Research/PA/pa196-vs-u208/PA-BRIDGE.md.

[4] ProveIt incoming research. pa_consistency_comparison_and_new196_2026-11-13.zip, especially the 191-token diagonal calculus and axioms/diagonal196/. Archive SHA-256 6fcb1cf2878eda152beda0b17cc0242c003908fa8adf07c81a2bbf49870f1866. https://github.com/VladimirReshetnikov/ProveIt/blob/906367250a7f96c923147b10df392294bced7be5/docs/incoming/pa_consistency_comparison_and_new196_2026-11-13.zip.

[5] ProveIt. All 27 incoming PA research archives at the pinned commit. https://github.com/VladimirReshetnikov/ProveIt/tree/906367250a7f96c923147b10df392294bced7be5/docs/incoming. The accompanying inventory records every inspected archive and member.

[6] ProveIt. ZFC small-machine research and inherited register-engine lineage. https://github.com/VladimirReshetnikov/ProveIt/tree/906367250a7f96c923147b10df392294bced7be5/Computability/BusyBeaver/Research/ZFC.

[7] ProveIt. *ConciseIPRA*. https://github.com/VladimirReshetnikov/ProveIt/tree/906367250a7f96c923147b10df392294bced7be5/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.